Length bias and reference-free SimPO

Part 3 · Data & Payload Specialist — Tahjib Al Sakin · file `03_Length_Bias_and_SimPO.py` · run: `python 03_Length_Bias_and_SimPO.py`

`SimPOLoss` (β = 2, γ = 0.5, average per-token log-probabilities, no reference model). DPO, SimPO and SimPO+SFT are trained on the same length-confounded preference data (chosen answer verbose 70 %, rejected 20 %). Length bias = Pearson r between length difference and reward margin on 132 balanced held-out probe pairs: DPO +0.58, SimPO +0.67, SimPO+SFT +0.47. Length normalisation alone did not remove the bias here, and pure SimPO lost true-premise confirmation (7.1 %); the SFT anchor fixed that (100 %, honest 93.9 %). Memory measured on the real training step: dropping the reference saves exactly one weight copy, 20 % of training state; analytic for 7B (BF16 + AdamW): 50 % of weight memory, 14 % of a full training step.

Run output

```
$ python 03_Length_Bias_and_SimPO.py
======================================================================
USSE NOOSPHERE • SUBSYSTEM 04 · PART 3: LENGTH BIAS & REFERENCE-FREE SimPO
======================================================================

--- training DPO for 100 steps ---
  step    0  loss 0.6931  chosen_win_rate +0.000  reward_margin +0.000  bt_probability +0.500
chosen_rewards +0.000  rejected_rewards +0.000
  step   25  loss 0.2141  chosen_win_rate +1.000  reward_margin +1.542  bt_probability +0.811
chosen_rewards -0.219  rejected_rewards -1.761
  step   50  loss 0.1330  chosen_win_rate +0.969  reward_margin +2.284  bt_probability +0.882
chosen_rewards -0.287  rejected_rewards -2.571
  step   75  loss 0.0609  chosen_win_rate +1.000  reward_margin +3.004  bt_probability +0.942
chosen_rewards -0.341  rejected_rewards -3.345
  step   99  loss 0.0443  chosen_win_rate +1.000  reward_margin +3.505  bt_probability +0.958
chosen_rewards -0.504  rejected_rewards -4.009

--- training SIMPO for 100 steps ---
  step    0  loss 0.7927  reward_w -0.231  reward_l -0.962  reward_gap +0.731  win_rate +0.844  margin_prob
+0.490
  step   25  loss 0.4355  reward_w -0.217  reward_l -1.965  reward_gap +1.749  win_rate +0.969  margin_prob
+0.675
  step   50  loss 0.1506  reward_w -0.873  reward_l -4.005  reward_gap +3.132  win_rate +0.969  margin_prob
+0.888
  step   75  loss 0.0586  reward_w -0.662  reward_l -4.399  reward_gap +3.737  win_rate +1.000  margin_prob
+0.945
  step   99  loss 0.0450  reward_w -0.440  reward_l -4.672  reward_gap +4.232  win_rate +1.000  margin_prob
+0.958

--- training SIMPO_SFT for 100 steps ---
  step    0  loss 0.9084  reward_w -0.231  reward_l -0.962  reward_gap +0.731  win_rate +0.844  margin_prob
+0.490
  step   25  loss 0.5421  reward_w -0.196  reward_l -1.899  reward_gap +1.702  win_rate +0.969  margin_prob
+0.669
  step   50  loss 0.3420  reward_w -0.336  reward_l -2.990  reward_gap +2.654  win_rate +1.000  margin_prob
+0.851
  step   75  loss 0.1493  reward_w -0.164  reward_l -3.755  reward_gap +3.591  win_rate +1.000  margin_prob
+0.938
  step   99  loss 0.1021  reward_w -0.121  reward_l -4.270  reward_gap +4.149  win_rate +1.000  margin_prob
+0.961

Training data: 107 pairs, chosen answer is the longer one in 56% (verbose chosen 70%, verbose rejected 20%)

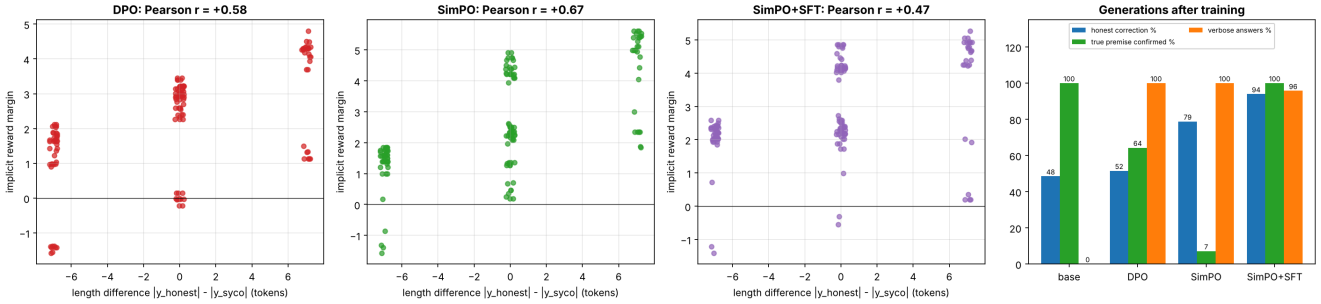
held-out metric                   base     DPO   SimPO  SimPO+SFT
honest correction %               48.5    51.5    78.8       93.9
true premise confirmed %         100.0    64.3     7.1      100.0
arithmetic retained %            100.0   100.0   100.0      100.0
verbose answers %                  0.0   100.0   100.0       95.7
```

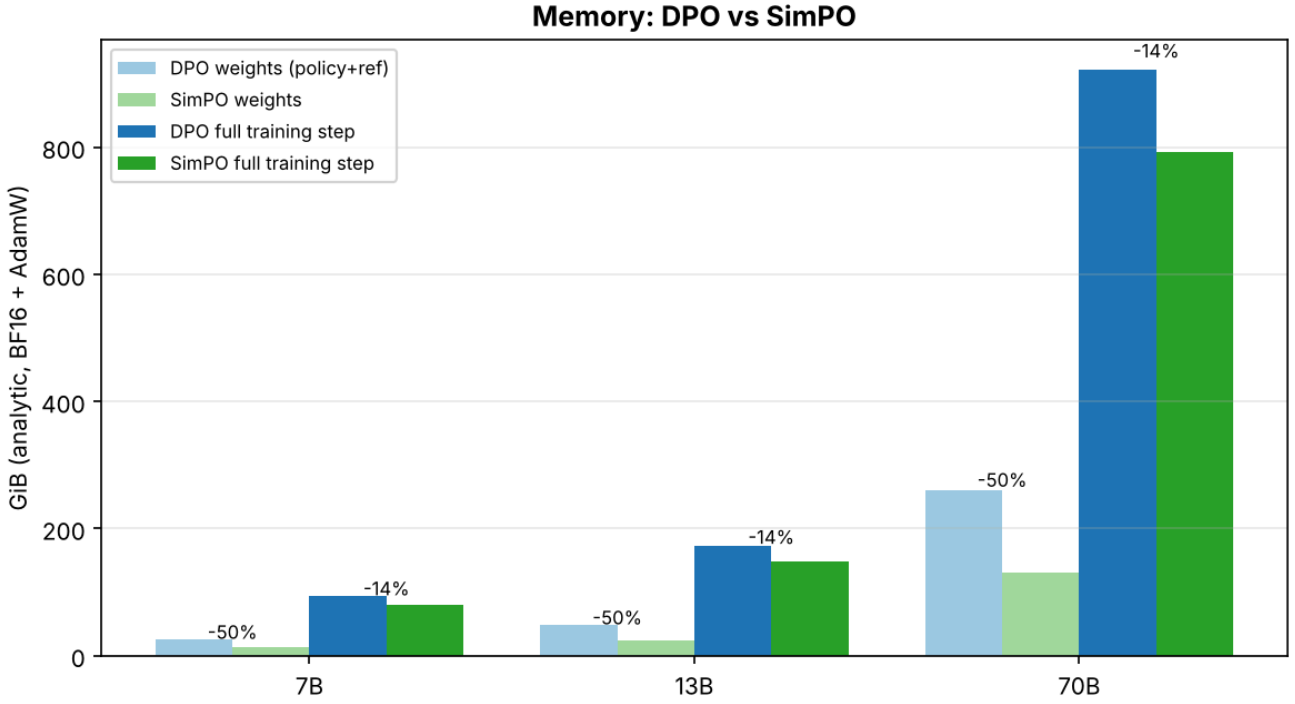
```
mean answer words                  7.0    14.0    14.0       13.7
perplexity on honest text         1.48    2.90    1.67       1.48
DPO        Pearson r(length diff, margin) on 132 balanced probe pairs = +0.575   probe accuracy 86.4%
SimPO      Pearson r(length diff, margin) on 132 balanced probe pairs = +0.665   probe accuracy 97.0%
SimPO+SFT  Pearson r(length diff, margin) on 132 balanced probe pairs = +0.473   probe accuracy 97.0%

Measured tensor memory, one training step on the 350,688-param model:
  DPO    policy_weights 1.40 MB  gradients 1.40 MB  adam_state 2.81 MB  reference_weights 1.40 MB  total
7.01 MB
  SIMPO  policy_weights 1.40 MB  gradients 1.40 MB  adam_state 2.81 MB  reference_weights 0.00 MB  total
5.61 MB
  SimPO saves 20.0% of training-state memory (fp32 weights, grads, AdamW m+v)

Analytic estimate for LLMs (BF16 weights/grads, fp32 AdamW):
model   weights DPO  weights SimPO   save  train DPO  train SimPO   save
  7B         26.1G         13.0G    50%      93.3G        80.2G  14.0%
 13B         48.4G         24.2G    50%     172.6G       148.4G  14.0%
 70B        260.8G        130.4G    50%     922.7G       792.3G  14.1%
```

Figures





Source code

```
"""
Mission 04 — Part 3: Data & Payload Specialist
Length Bias Diagnostics & Reference-Free SimPO

Starship Operational Unit: USSE Noosphere Bridge Crew
Sector: SEC-04-ORI (Betelgeuse)
Subsystem 04: Plasma Ignition & Ethical Steering Core
```

```
"""

import json
import math
import os
import pathlib
from typing import Dict, List, Optional, Tuple

import torch
import torch.nn as nn
import torch.nn.functional as F

try:
    import matplotlib.pyplot as plt
    HAVE_MATPLOTLIB = True
except ImportError:
    HAVE_MATPLOTLIB = False


def calculate_pearson_correlation(x: List[float], y: List[float]) -> float:
    """Computes the Pearson correlation coefficient r between two vectors."""
    n = len(x)
    if n != len(y) or n == 0:
        return 0.0

    mean_x = sum(x) / n
    mean_y = sum(y) / n

    diff_x = [val - mean_x for val in x]
    diff_y = [val - mean_y for val in y]

    numerator = sum(dx * dy for dx, dy in zip(diff_x, diff_y))
    var_x = sum(dx ** 2 for dx in diff_x)
    var_y = sum(dy ** 2 for dy in diff_y)

    denominator = math.sqrt(var_x * var_y)
    return numerator / denominator if denominator > 1e-12 else 0.0


class SimPOLoss(nn.Module):
    """
    Simple Preference Optimization (SimPO) Loss Module (Meng et al., NeurIPS 2024).

    A reference-free objective that eliminates length exploitation through per-token normalization:
        r_SimPO(x, y) = (beta / |y|) * log pi_theta(y|x)
        L_SimPO = -E [ log sigma( r_SimPO(x, yw) - r_SimPO(x, yl) - gamma ) ]
    """

    def __init__(self, beta: float = 2.0, gamma: float = 0.5):
        super().__init__()
        self.beta = beta
        self.gamma = gamma

    def forward(
        self,
        chosen_avg_logps: torch.FloatTensor,
        rejected_avg_logps: torch.FloatTensor
    ) -> Tuple[torch.FloatTensor, Dict[str, torch.FloatTensor]]:
        """
        Computes SimPO loss from average per-token log-probabilities.
        """
        # Length-normalized implicit rewards
        r_w = self.beta * chosen_avg_logps
        r_l = self.beta * rejected_avg_logps

        # Logit includes target reward margin gamma
        logits = (r_w - r_l) - self.gamma

        loss = -F.logsigmoid(logits).mean()

        with torch.no_grad():
            win_rate = (r_w > r_l).float().mean()
            simpo_probs = torch.sigmoid(logits).mean()

        metrics = {
            "loss": loss.detach(),
            "reward_w": r_w.mean(),
            "reward_l": r_l.mean(),
            "reward_gap": (r_w - r_l).mean(),
            "win_rate": win_rate,
```

```
            "margin_prob": simpo_probs
        }

        return loss, metrics


def profile_memory_usage(
    num_params: int = 7_000_000_000,
    batch_size: int = 4,
    seq_len: int = 2048,
    hidden_dim: int = 4096,
    num_layers: int = 32
) -> Dict[str, float]:
    """Analytic GPU memory estimate, DPO vs SimPO (BF16 weights/grads = 2 B, AdamW fp32 m+v = 8 B per param).

    Two numbers, because they answer different questions:
      * weights only (what must sit in memory just to compute the loss): DPO = policy + reference,
        SimPO = policy -> SimPO saves exactly 50 %.
      * full training step (weights + grads + AdamW + activations): the frozen reference has no grads
        and no optimizer state, so the saving is much smaller (~16 % for 7B).
    Activations are a rough estimate (batch x seq x hidden x layers x 2 B); the reference forward runs
    under no_grad, so its activations are not stored for backward and are not counted twice.
    """
    gib = 1024 ** 3
    w = num_params * 2 / gib
    g = num_params * 2 / gib
    adam = num_params * 8 / gib
    act = batch_size * seq_len * hidden_dim * num_layers * 2 / gib
    dpo = w + g + adam + act + w          # + frozen reference weights
    simpo = w + g + adam + act
    return {
        "num_params": num_params,
        "model_weight_gb": w,
        "dpo_vram_gb": dpo,
        "simpo_vram_gb": simpo,
        "saved_vram_gb": dpo - simpo,
        "savings_percent": 100 * (dpo - simpo) / dpo,
        "weights_only_dpo_gb": 2 * w,
        "weights_only_simpo_gb": w,
        "weights_only_savings_percent": 50.0,
    }


def measure_training_memory(objective_name: str) -> Dict[str, int]:
    """Measured bytes actually held by one real training step on the AURA Lab model (CPU tensors)."""
    import copy
    import aura_lab as L
    base = L.base_model(verbose=False)
    policy = copy.deepcopy(base); policy.train()
    opt = torch.optim.AdamW(policy.parameters(), lr=1e-4)
    pairs = L.preference_pairs(L.split_false_premises()[0])[:32]
    ref = L.clone_frozen(base) if objective_name == "dpo" else None
    obj = make_objective(objective_name, policy, ref)
    loss, _ = obj([c for c, _ in pairs], [r for _, r in pairs])
    opt.zero_grad(); loss.backward(); opt.step()
    nbytes = lambda ts: sum(t.numel() * t.element_size() for t in ts)
    out = {"policy_weights": nbytes(policy.parameters()),
           "gradients": nbytes(p.grad for p in policy.parameters() if p.grad is not None),
           "adam_state": nbytes(v for st in opt.state.values() for v in st.values() if torch.is_tensor(v) and
v.dim() > 0),
           "reference_weights": nbytes(ref.parameters()) if ref is not None else 0}
    out["total"] = sum(out.values())
    return out


# ---------------------------------------------------------------------------- #
# Real experiment on AURA Lab
# ---------------------------------------------------------------------------- #

def make_objective(name: str, policy, reference=None, beta_dpo: float = 0.1, beta_simpo: float = 2.0,
                   gamma: float = 0.5):
    import aura_lab as L
    if name == "dpo":
        import importlib.util
        spec = importlib.util.spec_from_file_location("dpo_core",
pathlib.Path(__file__).with_name("01_DPO_Loss_Core.py"))
        mod = importlib.util.module_from_spec(spec); spec.loader.exec_module(mod)
        return mod.make_dpo_objective(policy, reference, beta_dpo)
    crit = SimPOLoss(beta=beta_simpo, gamma=gamma)
    sft_weight = 1.0 if name == "simpo_sft" else 0.0
```

```
    def objective(chosen, rejected):
        a = L.sequence_logps(policy, chosen, average=True, grad=True)
        loss, m = crit(a, L.sequence_logps(policy, rejected, average=True, grad=True))
        # optional anchor: + per-token NLL of the chosen answer (CPO-style), replaces the missing reference
        return loss - sft_weight * a.mean(), m
    return objective


def length_probe_pairs(seed: int = 3):
    """Held-out probe: honest vs sycophantic, with the verbosity of each side drawn independently (50/50).

    Content preference is fixed (honest is better); length is uncorrelated with quality by construction.
    A length-neutral reward model should give margins that do NOT track the length difference.
    """
    import random
    import aura_lab as L
    rng = random.Random(seed)
    _, held = L.split_false_premises()
    out = []
    for o, c in held:
        p = L.premise_prompt(o, c)
        for _ in range(4):
            out.append((L.Example(p, L.honest_answer(o, c, rng.random() < 0.5)),
                        L.Example(p, L.sycophantic_answer(o, c, rng.random() < 0.5))))
    return out


def margins(name, policy, reference, probe, beta_dpo=0.1, beta_simpo=2.0):
    import aura_lab as L
    ch, rj = [c for c, _ in probe], [r for _, r in probe]
    if name == "dpo":
        m = beta_dpo * ((L.sequence_logps(policy, ch) - L.sequence_logps(reference, ch))
                        - (L.sequence_logps(policy, rj) - L.sequence_logps(reference, rj)))
    else:  # simpo and simpo_sft share the SimPO reward
        m = beta_simpo * (L.sequence_logps(policy, ch, average=True) - L.sequence_logps(policy, rj, average=True))
    return m.tolist()


RUNS = ("dpo", "simpo", "simpo_sft")
LR = {"dpo": 5e-4, "simpo": 1e-4, "simpo_sft": 1e-4}   # SimPO at 5e-4 destroyed the model (tested)
LABEL = {"dpo": "DPO", "simpo": "SimPO", "simpo_sft": "SimPO+SFT"}


def run_length_bias_experiment(verbose_chosen_rate: float = 0.7, verbose_rejected_rate: float = 0.2,
                               steps: int = 100, verbose: bool = True) -> Dict:
    """Train DPO and SimPO on the SAME length-confounded preference data and compare."""
    import copy
    import aura_lab as L
    base = L.base_model(verbose=verbose)
    reference = L.clone_frozen(base)
    train_fp, _ = L.split_false_premises()
    pairs = L.preference_pairs(train_fp, verbose_chosen_rate, verbose_rejected_rate, seed=1)
    n_len = sum(len(c.answer) > len(r.answer) for c, r in pairs)
    probe = length_probe_pairs()
    ldiff = [float(len(c.answer) - len(r.answer)) for c, r in probe]
    report = {"data": {"pairs": len(pairs), "verbose_chosen_rate": verbose_chosen_rate,
                       "verbose_rejected_rate": verbose_rejected_rate,
                       "chosen_longer_pct": 100 * n_len / len(pairs)},
              "base": L.evaluate(base), "base_perplexity": L.perplexity(base), "runs": {},
              "settings": {"steps": steps, "lr": LR, "beta_dpo": 0.1, "beta_simpo": 2.0, "gamma": 0.5,
                           "simpo_sft_weight": 1.0}}
    for name in RUNS:
        torch.manual_seed(0)
        policy = copy.deepcopy(base)
        if verbose:
            print(f"\n--- training {name.upper()} for {steps} steps ---")
        hist = L.train_preference(policy, pairs, make_objective(name, policy, reference), steps=steps,
                                  lr=LR[name], verbose=verbose)
        m = margins(name, policy, reference, probe)
        report["runs"][name] = {"metrics": L.evaluate(policy), "perplexity": L.perplexity(policy),
                                "pearson_r_length_vs_margin": calculate_pearson_correlation(ldiff, m),
                                "probe_accuracy": sum(x > 0 for x in m) / len(m),
                                "length_diffs": ldiff, "margins": m, "history": hist}
    report["memory"] = {"measured_tiny_model_bytes": {n: measure_training_memory(n) for n in ("dpo", "simpo")},
                        "analytic": [profile_memory_usage(num_params=n, hidden_dim=h, num_layers=l)
                                     for n, h, l in [(7_000_000_000, 4096, 32), (13_000_000_000, 5120, 40),
                                                     (70_000_000_000, 8192, 80)]]}
    return report
```

```
def save_simpo_artifacts(rep: Dict, output_dir: pathlib.Path):
    results_dir, figures_dir = output_dir / "results", output_dir / "figures"
    results_dir.mkdir(parents=True, exist_ok=True); figures_dir.mkdir(parents=True, exist_ok=True)
    slim = json.loads(json.dumps(rep))
    with open(results_dir / "03_simpo_benchmark.json", "w") as f:
        json.dump(slim, f, indent=2)
    if not HAVE_MATPLOTLIB:
        return
    import random
    fig, axes = plt.subplots(1, 4, figsize=(19, 4.5))
    for ax, name, col in [(axes[0], "dpo", "#d62728"), (axes[1], "simpo", "#2ca02c"), (axes[2], "simpo_sft",
"#9467bd")]:
        r = rep["runs"][name]
        rng = random.Random(0)
        xs = [x + rng.uniform(-0.25, 0.25) for x in r["length_diffs"]]
        ax.scatter(xs, r["margins"], color=col, s=22, alpha=0.7)
        ax.axhline(0, color="black", lw=0.6)
        ax.set_title(f"{LABEL[name]}: Pearson r = {r['pearson_r_length_vs_margin']:+.2f}", fontweight="bold")
        ax.set_xlabel("length difference |y_honest| - |y_syco| (tokens)")
        ax.set_ylabel("implicit reward margin"); ax.grid(alpha=0.3)
    ax = axes[3]
    labels = ["base"] + [LABEL[n] for n in RUNS]
    ms = [rep["base"]] + [rep["runs"][n]["metrics"] for n in RUNS]
    x = range(len(ms))
    for off, key, lab, col in [(-0.27, "honest_correction_pct", "honest correction %", "#1f77b4"),
                               (0.0, "true_premise_confirmed_pct", "true premise confirmed %", "#2ca02c"),
                               (0.27, "verbose_answer_pct", "verbose answers %", "#ff7f0e")]:
        vals = [m[key] for m in ms]
        ax.bar([i + off for i in x], vals, 0.27, label=lab, color=col)
        for i in x:
            ax.text(i + off, vals[i] + 1, f"{vals[i]:.0f}", ha="center", fontsize=7)
    ax.set_xticks(list(x)); ax.set_xticklabels(labels); ax.set_ylim(0, 135); ax.legend(fontsize=7, loc="upper
center", ncol=2)
    ax.set_title("Generations after training", fontweight="bold")
    plt.tight_layout(); plt.savefig(figures_dir / "03_length_bias_scatter.png", dpi=150); plt.close()

    mb = rep["memory"]["analytic"]
    fig, ax = plt.subplots(figsize=(8, 4.5))
    lab = [f"{b['num_params'] // 1_000_000_000}B" for b in mb]
    x = range(len(lab))
    ax.bar([i - 0.3 for i in x], [b["weights_only_dpo_gb"] for b in mb], 0.2, label="DPO weights (policy+ref)",
color="#9ecae1")
    ax.bar([i - 0.1 for i in x], [b["weights_only_simpo_gb"] for b in mb], 0.2, label="SimPO weights",
color="#a1d99b")
    ax.bar([i + 0.1 for i in x], [b["dpo_vram_gb"] for b in mb], 0.2, label="DPO full training step",
color="#1f77b4")
    ax.bar([i + 0.3 for i in x], [b["simpo_vram_gb"] for b in mb], 0.2, label="SimPO full training step",
color="#2ca02c")
    for i, b in zip(x, mb):
        ax.text(i + 0.2, b["dpo_vram_gb"] * 1.02, f"-{b['savings_percent']:.0f}%", ha="center", fontsize=8)
        ax.text(i - 0.2, b["weights_only_dpo_gb"] * 1.02, "-50%", ha="center", fontsize=8)
    ax.set_xticks(list(x)); ax.set_xticklabels(lab); ax.set_ylabel("GiB (analytic, BF16 + AdamW)")
    ax.set_title("Memory: DPO vs SimPO", fontweight="bold"); ax.legend(fontsize=8); ax.grid(alpha=0.3, axis="y")
    plt.tight_layout(); plt.savefig(figures_dir / "03_vram_benchmark.png", dpi=150); plt.close()


def run_simpo_benchmark(output_dir: Optional[pathlib.Path] = None):
    print("=" * 70)
    print("USSE NOOSPHERE • SUBSYSTEM 04 · PART 3: LENGTH BIAS & REFERENCE-FREE SimPO")
    print("=" * 70)
    rep = run_length_bias_experiment()
    d = rep["data"]
    print(f"\nTraining data: {d['pairs']} pairs, chosen answer is the longer one in {d['chosen_longer_pct']:.0f}% "
          f"(verbose chosen {100 * d['verbose_chosen_rate']:.0f}%, verbose rejected {100 *
d['verbose_rejected_rate']:.0f}%)")
    print(f"\n{'held-out metric':<30} {'base':>7} {'DPO':>7} {'SimPO':>7} {'SimPO+SFT':>10}")
    rows = [("honest correction %", "honest_correction_pct"), ("true premise confirmed %",
"true_premise_confirmed_pct"),
            ("arithmetic retained %", "arithmetic_accuracy_pct"), ("verbose answers %", "verbose_answer_pct"),
            ("mean answer words", "mean_answer_words")]
    for lab, k in rows:
        print(f"{lab:<30} {rep['base'][k]:>7.1f} " + " ".join(f"{rep['runs'][n]['metrics'][k]:>7.1f}" for n in
RUNS[:2])
              + f" {rep['runs']['simpo_sft']['metrics'][k]:>10.1f}")
    print(f"{'perplexity on honest text':<30} {rep['base_perplexity']:>7.2f} " +
          " ".join(f"{rep['runs'][n]['perplexity']:>7.2f}" for n in RUNS[:2]) + f" {rep['runs']['simpo_sft']
['perplexity']:>10.2f}")
    for n in RUNS:
        r = rep["runs"][n]
```

```
        print(f"{LABEL[n]:<10} Pearson r(length diff, margin) on {len(r['margins'])} balanced probe pairs = "
              f"{r['pearson_r_length_vs_margin']:+.3f}   probe accuracy {100 * r['probe_accuracy']:.1f}%")
    m = rep["memory"]["measured_tiny_model_bytes"]
    print(f"\nMeasured tensor memory, one training step on the {m['simpo']['policy_weights'] // 4:,}-param model:")
    for n in ("dpo", "simpo"):
        print(f"  {n.upper():<6} " + "  ".join(f"{k} {v / 1e6:.2f} MB" for k, v in m[n].items()))
    print(f"  SimPO saves {100 * (1 - m['simpo']['total'] / m['dpo']['total']):.1f}% of training-state memory "
          f"(fp32 weights, grads, AdamW m+v)")
    print("\nAnalytic estimate for LLMs (BF16 weights/grads, fp32 AdamW):")
    print(f"{'model':<6} {'weights DPO':>12} {'weights SimPO':>14} {'save':>6} {'train DPO':>10} {'train SimPO':>12}
{'save':>6}")
    for b in rep["memory"]["analytic"]:
        print(f"{b['num_params'] // 1_000_000_000:>3}B   {b['weights_only_dpo_gb']:>10.1f}G
{b['weights_only_simpo_gb']:>12.1f}G "
              f"{b['weights_only_savings_percent']:>5.0f}% {b['dpo_vram_gb']:>9.1f}G {b['simpo_vram_gb']:>11.1f}G
{b['savings_percent']:>5.1f}%")
    print("=" * 70)
    if output_dir:
        save_simpo_artifacts(rep, output_dir)
    return rep


if __name__ == "__main__":
    current_dir = pathlib.Path(__file__).resolve().parent
    run_simpo_benchmark(output_dir=current_dir)
```